

System Design & Architecture Masterclass

Complete Chapter-by-Chapter LinkedIn Posts & Growth Catalog

■ ■ Grokking the System Design Interview

Post #01: The 45-Minute 7-Step FAANG Interview Framework

[Target Chapter: 01 System Design Interview Basics & Preparation.html](#)

■ ■ How Staff & Principal engineers structure the 45-minute System Design Interview to get L6/L7 offers:

Most candidates fail System Design Interviews not because they don't know distributed components, but because they fail to pace the 45-minute window.

Here is the exact time allocation used by FAANG interviewers:

■ 00:00 - 05:00 | Step 1: Requirements Scope

Clarify 3-4 core Functional use cases & establish Non-Functional SLOs (Latency < 100ms P99, 99.99% Availability).

■ 05:00 - 10:00 | Step 2: Back-of-the-Envelope Math

DAU -> QPS (Read/Write ratio) -> Storage per year -> Network Bandwidth -> 80/20 RAM Cache rule.

■ 10:00 - 15:00 | Step 3: APIs & Data Model

Define REST/gRPC contracts with status codes and SQL/NoSQL schema DDL with partition keys.

■ 15:00 - 25:00 | Step 4: High-Level Architecture

Draw the baseline end-to-end data flow: Client -> CDN/LB -> API Gateway -> Services -> Cache -> DB -> Async Message Bus.

■ 25:00 - 38:00 | Step 5 & 6: Component Deep Dives

Detailed algorithms (QuadTree, SkipList, Vector Clocks), concurrency control, and replication protocols.

■ 38:00 - 45:00 | Step 7: Trade-offs & Fault Tolerance

SPOF elimination, CAP theorem trade-offs, rate limiting, disaster recovery, and interviewer curveballs.

■ Which phase of the 45-minute SDI do you find most stressful? Let me know below!

(■ Full live guide in first comment! ■)

#SystemDesign #CodingInterview #FAANG #SoftwareEngineering #TechCareers #Architecture

Post #02: Consistent Hashing with Virtual Nodes & Minimum Disruption

[Target Chapter: 02 Core Distributed System Building Blocks.html](#)

■ Why naive hash mod ($\text{hash}(\text{key}) \% N$) breaks horizontal scaling in caching clusters:

When a cache cluster has N servers, using $\text{hash}(\text{key}) \% N$ means that when 1 server crashes or is added, almost ALL keys (~100%) rehash to different servers, causing a catastrophic cache stampede on the primary database!

■ The Solution: Consistent Hashing on a 2^{128} Token Ring.

1■ ■ Ring Topology: Both cache servers and data keys hash onto the same circular ring space $[0, 2^{128} - 1]$.

2■ ■ Clockwise Routing: A key is assigned to the first server whose token is $\geq$ the key's hash.

3■ ■ Minimum Disruption: When a server is added or removed, ONLY keys between adjacent nodes move (average K/N keys relocated instead of 100%).

4■ ■ Virtual Nodes: Assigning 150-200 virtual tokens per physical server reduces load distribution variance below $\sigma \approx 1/\sqrt{V} \approx 7\%$.

(Full Python & TypeScript consistent hashing ring implementation in the guide!)

■ Have you used Consistent Hashing in Redis Cluster, Cassandra, or Memcached?

(■ Full live guide in first comment! ■)

#ConsistentHashing #Caching #DistributedSystems #Redis #SystemDesign #SoftwareArchitecture

Post #03: PACELC Theorem: Beyond the Simplistic CAP Theorem

[Target Chapter: 03 System Design Concepts & Tradeoffs.html](#)

■ ■ Why the classic CAP Theorem is incomplete for real-world distributed databases:

Every engineer knows CAP: In presence of Partition (P), choose Availability (A) or Consistency (C). But what happens when the network is working normally (NO partition)?

- That's why Daniel Abadi formulated the PACELC Theorem:
- If Partition (P): choose Availability (A) OR Consistency (C).
- ELSE (E): choose Latency (L) OR Consistency (C).

Real Database Classifications:

- PA/EL (Amazon DynamoDB, Apache Cassandra):

During partition, stays Available (PA). Under normal operation, optimizes for Sub-10ms Latency (EL) using eventual consistency.

- PC/EC (Google Spanner, CockroachDB):

During partition, preserves Consistency (PC). Under normal operation, preserves strict Consistency (EC) via multi-phase consensus (Raft/Paxos/TrueTime) at the cost of higher latency.

- MongoDB: PC/EC by default, tunable to PA/EL.

- Do your production databases prioritize Latency (EL) or Consistency (EC)?

(■ Full live guide in first comment! ■)

#Databases #CAPTheorem #DistributedSystems #NoSQL #SystemDesign #BackendEngineering

Post #04: REST vs gRPC vs WebSocket vs Server-Sent Events (SSE)

[Target Chapter: 04 Architectural Comparison & Pattern Tradeoffs.html](#)

- Choosing the right client-server communication protocol in distributed architectures:

Picking the wrong communication protocol can cause connection exhaustion, high CPU overhead, or head-of-line blocking. Here is the decision matrix:

1■■ REST over HTTP/1.1 or HTTP/2:

- Best for: Public APIs, CRUD endpoints, web clients.
- Strengths: Universally supported, human-readable JSON, built-in edge caching.
- Weaknesses: High payload overhead, text-based serialization.

2■■ gRPC over HTTP/2:

- Best for: Internal Microservices east-west communication.
- Strengths: Protocol Buffers binary serialization (up to 7x faster than JSON), bidirectional streaming, strict typed contracts.
- Weaknesses: Harder to inspect in browser dev tools without proxy (gRPC-Web).

3■■ WebSockets:

- Best for: True bidirectional real-time communication (multiplayer games, chat, Figma canvas sync).
- Strengths: Full-duplex persistent TCP connection with minimal 2-byte framing overhead.
- Weaknesses: Stateful connections require sticky load balancing and complex horizontal scaling.

4■■ Server-Sent Events (SSE):

- Best for: Unidirectional server-to-client streaming (ChatGPT token generation, stock tickers, notification feeds).
- Strengths: Runs over standard HTTP/2, built-in automatic client reconnection.

- What protocol do you use for AI streaming: SSE or WebSockets?

(■ Full live guide in first comment! ■)

#API #gRPC #WebSockets #REST #Microservices #SystemDesign #SoftwareEngineering

Post #05: Designing TinyURL: Base62 vs Key Generation Service (KGS)

[Target Chapter: 05 Classic System Design Problems \(Part 1\).html](#)

- Why MD5/SHA-256 hashing fails for TinyURL and how a Key Generation Service (KGS) fixes it:

Designing a URL Shortener seems simple until you hit 100,000,000 URLs/day and need zero collision guarantees.

- The Flawed Approach: MD5(URL) -> Base62

Hashing a long URL gives 128 bits. Taking the first 7 characters causes hash collisions. Appending user IDs or random salts requires continuous database collision checks, causing high latency write bottlenecks!

- The Staff-Level Solution: Dedicated Key Generation Service (KGS)

1■■ Pre-generate 7-character Base62 keys (\$62^7 \approx 3.52 \times 10^{12}\$ unique keys).

2■■ Store keys in two tables: `Used\_Keys` and `Available\_Keys`.

3■■ KGS server loads 5,000 available keys into RAM.

4■■ When a request arrives, KGS hands out a key in < 1 millisecond with ZERO database hashing or collision checks!

5■■ Redundant KGS replicas with master-standby leases ensure no duplicate keys are ever handed out.

(Includes complete Python + TypeScript production code with Redis caching!)

■ How would you handle key recovery when shortened URLs expire?

(■ Full live guide in first comment! ■)

#TinyURL #SystemDesign #Algorithms #CodingInterview #DistributedSystems #SoftwareArchitecture

Post #06: Designing Uber Backend: H3 Hexagons & Dynamic Surge Multiplier

[Target Chapter: 06 Classic System Design Problems \(Part 2\).html](#)

■ How Uber matches riders to nearby drivers in < 2 seconds across 10,000+ global cities:

Traditional SQL spatial queries (``ST_DWithin`` on latitude/longitude) degrade exponentially when ingesting 1,000,000 GPS updates every 4 seconds.

■ The Uber Architecture:

1■ Uber H3 Hierarchical Spatial Index:

Partitions city terrain into uniform hexagons (Resolution 7 to 9). Unlike squares, hexagons have identical distances to all 6 neighboring cells, preventing directional distortion!

2■ Ingestion Pipeline:

Drivers stream GPS pings over persistent WebSockets -> Ingested into Redis Geospatial (``GEOADD``) with 30-second TTLs.

3■ Dynamic Surge Pricing Engine:

Computes the supply-demand ratio per hexagon cell:

$$\text{Ratio} = \frac{\text{Rider App Searches}}{\text{Active Available Drivers}}$$

If $\text{Ratio} > 1.25$, apply exponential multiplier capped at 3.0x to balance local ride requests.

4■ Atomic Trip Dispatch:

Uses distributed Redis locks (``SETNX trip_lock EX 5``) to prevent duplicate dispatches to the same driver.

■ Have you worked with Geospatial indexing: H3, S2 Geometry, or QuadTrees?

(■ Full live guide in first comment! ■)

#Uber #Geospatial #Algorithms #SystemDesign #Redis #SoftwareEngineering

Post #07: Designing Double-Entry Banking Ledgers & Stripe-Style Idempotency

[Target Chapter: 07 Advanced & Medium System Design Problems.html](#)

■ How Stripe and modern fintech ledgers guarantee zero money creation/loss under network failures:

In financial systems, a single lost penny or duplicate charge can result in catastrophic compliance failure.

Here is how modern banking ledgers maintain 100% mathematical integrity:

1■ Double-Entry Accounting Invariant:

Money is never created or destroyed; it only moves between accounts.

Every transaction MUST contain balanced Debit and Credit legs:

$$\sum \text{Debits} = \sum \text{Credits}$$

2■ Idempotency Key Engine:

Client sends header ``Idempotency-Key: uuid-v4``.

- API Gateway checks Redis (``SETNX lock:key EX 120``).

- If in-flight, return ``409 Conflict`` or wait.

- If completed, return cached response directly from Redis without hitting the payment rail!

3■ Immutable Append-Only Ledger:

Ledger records are NEVER updated or deleted (``UPDATE`/`DELETE`` forbidden at SQL grant level). Corrections are applied via reversing debit/credit journal entries.

4■ Cryptographic Audit Trail:

Each journal entry stores a SHA-256 hash chaining back to the previous entry: ``Hash_N = SHA256(Hash_{N-1} + EntryData)``.

■ How do you handle race conditions in payment gateways: Optimistic Locking or Redis Redlock?

(■ Full live guide in first comment! ■)

#FinTech #Banking #Stripe #Idempotency #SystemDesign #PostgreSQL #SoftwareEngineering

Post #08: Designing ChatGPT at 100M Scale: Token Streaming & Radix KV-Cache

[Target Chapter: 08 Flagship AI & LLM System Design Problems.html](#)

■ The complete production architecture of ChatGPT serving 100M daily active users:

Serving large language models (LLMs) is fundamentally different from traditional REST APIs because generation is

memory-bandwidth bound and token-by-token sequential.

■ Core System Architecture:

1■■ WebSocket Full-Duplex Gateway:

Maintains persistent bi-directional streams. Allows clients to send mid-stream abort signals (saving expensive GPU inference tokens).

2■■ RadixAttention (vLLM) KV-Cache Tree:

Maintains a prefix tree of Key-Value attention tensors in GPU VRAM across multiple requests. Reusing shared system prompts slashes Time-To-First-Token (TTFT) by up to 95%!

3■■ Continuous Iteration-Level Batching:

Traditional batching waits for the slowest request in a batch to complete. Continuous batching evicts completed sequences and inserts new requests on every token iteration step!

4■■ Compute & Bandwidth Sizing:

At 100M DAU with 50 tokens/sec peak: Requires cluster sizing across 250,000 H100 GPUs with 3.35 TB/s HBM3 memory bandwidth and 2.56 Gbps network egress.

■ What LLM serving engine do you use: vLLM, TensorRT-LLM, or TGI?

(■ Full live guide in first comment! ■)

#ArtificialIntelligence #ChatGPT #LLM #MachineLearning #SystemDesign #GenerativeAI

Post #09: The 11 Legendary Distributed Systems Whitepapers Every Architect Must Know

[Target Chapter: 09 Distributed Systems Case Studies & Legendary Papers.html](#)

■ If you want to master System Design, read these 11 seminal distributed systems papers:

All modern cloud architectures (AWS, GCP, Kafka, Cassandra) are direct implementations of 11 legendary whitepapers:

1■■ Amazon Dynamo (2007): Sloppy Quorums, Consistent Hashing & Vector Clocks.

2■■ Apache Kafka (2011): Zero-Copy OS Page Cache & Distributed Commit Logs.

3■■ Paxos & Raft Consensus (1998/2014): Leader Election, Quorums & State Machine Replication.

4■■ Google Bigtable (2006): LSM-Trees, MemTable SkipLists, Immutable SSTables & Bloom Filters.

5■■ Gossip Protocol & SWIM (2002): Infection-style decentralized failure detection without bottlenecks.

6■■ Google Chubby (2006): Distributed Lock Leases & Sequencer Fencing Tokens.

7■■ Apache ZooKeeper & ZAB (2010): Atomic Broadcast & Ephemeral Sequential ZNodes.

8■■ Google MapReduce (2004): Split, Map, In-Memory Spill, Network Shuffle & Speculative Execution.

9■■ Google File System (GFS / HDFS) (2003): Single Master, 64MB Chunkservers & 3x Rack-Aware Replication.

■ Apache Cassandra (2010): Masterless P2P Gossip, Tunable Quorums (R+W>N) & Tombstone GC.

1■■1■■ Consistent Hashing (1997): K/N Minimum Disruption Token Rings.

(Full architectural deep-dives and code for all 11 papers in the guide!)

■ Which paper had the biggest impact on how you design software?

(■ Full live guide in first comment! ■)

#DistributedSystems #Whitepapers #SystemDesign #ApacheKafka #SoftwareArchitecture #CS

■■ Grokking System Design Fundamentals

Post #10: Layer 4 vs Layer 7 Load Balancing & Consistent Hashing

[Target Chapter: 02 Load Balancing.html](#)

■■ Layer 4 (Transport) vs Layer 7 (Application) Load Balancing: When to use which?

Load balancers are the first line of defense in scalable infrastructure, but confusing L4 vs L7 can cause severe performance bottlenecks.

■ Layer 4 Load Balancing (TCP/UDP):

- Works at IP & Port level without decrypting or inspecting packet payloads.
- Extreme throughput (millions of packets/sec via Linux IPVS / Maglev).
- Cannot inspect HTTP headers, cookies, or path routing.

■ Layer 7 Load Balancing (HTTP/HTTPS/gRPC):

- Decrypts TLS and parses the full HTTP request (Headers, URL path, Cookies, JSON body).
- Intelligent routing: `/api/v1/orders` -> Orders Service; `/api/v1/users` -> Users Service.
- Supports header-based rate limiting, JWT validation, and WebSocket sticky sessions.
- Higher CPU overhead due to TLS termination and packet parsing.

■ Production Best Practice:

Two-tier topology: Layer 4 ECMP/Maglev LB at the edge for high-speed packet distribution -> Fan out to a fleet of Layer 7 Envoy/NGINX gateways for application routing!

■ What is your preferred load balancer: Envoy, NGINX, HAProxy, or AWS ALB?

(■ Full live guide in first comment! ■)

#LoadBalancing #Networking #DevOps #SystemDesign #Infrastructure #Microservices

Post #11: Caching Topologies: Cache-Aside vs Write-Through vs Write-Behind

[Target Chapter: 06 Caching and CDN.html](#)

■ Cache-Aside, Write-Through, Write-Behind (Write-Back): Choosing the right caching strategy:

Caching is essential for sub-millisecond read latency, but picking the wrong write topology causes cache inconsistency or data loss!

1■■ Cache-Aside (Lazy Loading):

- Read: App checks Cache. If Miss, reads from DB and writes to Cache.
- Write: App writes directly to DB, then invalidates/evicts cache key.
- Best for: General read-heavy workloads where stale data can be avoided via eviction.

2■■ Write-Through:

- App writes to Cache; Cache synchronously writes to DB before acknowledging.
- Strength: Cache and DB are always consistent.
- Weakness: Write latency is higher (2 write hops).

3■■ Write-Behind (Write-Back):

- App writes to Cache; Cache immediately acknowledges and asynchronously batches writes to DB!
- Strength: Ultra-fast write performance (absorbs write spikes).
- Risk: If the cache node crashes before flushing to disk, recent writes are LOST!

■ What eviction policy do you use: LRU, LFU, or FIFO?

(■ Full live guide in first comment! ■)

#Caching #Redis #Performance #SystemDesign #SoftwareEngineering #Backend

Post #12: B+ Trees vs LSM-Trees: The Battle of Database Storage Engines

[Target Chapter: 11 Databases and Indexing.html](#)

■ B+ Trees vs LSM-Trees: Why PostgreSQL uses B+ Trees while Cassandra/RocksDB uses LSM-Trees:

The fundamental tradeoff in database storage engines comes down to Read Latency vs Write Throughput.

■ B+ Trees (RDBMS: PostgreSQL, MySQL InnoDB):

- Ordered tree structure where all data pointers live in leaf nodes linked horizontally.
- Strength: Fast point reads ($O(\log N)$) and rapid range scans.
- Weakness: Writes require random disk I/O to update scattered tree pages, causing write amplification.

■ LSM-Trees (Log-Structured Merge-Trees: Cassandra, RocksDB, ScyllaDB):

- Writes append sequentially to an in-memory `MemTable` (SkipList) and Write-Ahead Log (WAL).
- When full, flushes to disk as immutable `SSTables`.
- Background compaction merges SSTables and purges tombstones.
- Strength: Blazing-fast sequential write throughput ($O(1)$ append).
- Weakness: Reads must check MemTable + Bloom Filters + multiple SSTables.

■ If you're building a write-heavy telemetry/IoT platform, which storage engine do you choose?

(■ Full live guide in first comment! ■)

#Databases #PostgreSQL #Cassandra #DataStructures #SystemDesign #SoftwareArchitecture

■■ Grokking Microservices Design Patterns

Post #13: Circuit Breaker Pattern: Preventing Cascading Failures with Resilience4j

[Target Chapter: 06 Circuit Breaker Pattern.html](#)

■■ How the Circuit Breaker pattern prevents 1 failing microservice from taking down your entire company:

When downstream Service B slows down or fails, upstream Service A keeps waiting on HTTP timeouts, consuming worker threads until thread pool exhaustion causes Service A to crash too!

■ The Circuit Breaker State Machine:

1■■ CLOSED (Normal):

Requests flow normally. Tracks success/failure rate.

2■■ OPEN (Tripped):

When failure rate exceeds threshold (e.g. 50% over 20 requests), circuit opens immediately.

All subsequent requests FAIL FAST with cached fallback or default response without calling Service B!

3■■ HALF-OPEN (Testing Recovery):

After a configured timeout (e.g. 10s), allows a trial batch of 5 requests through.

- If they succeed -> Transition back to CLOSED.
- If they fail -> Transition back to OPEN for another backoff period.

(Includes resilience patterns with Exponential Backoff and Full Jitter!)

■ Do you configure Circuit Breakers in code (Resilience4j) or at the Service Mesh layer (Envoy)?

(■ Full live guide in first comment! ■)

#Microservices #Resilience #CircuitBreaker #DevOps #SystemDesign #SoftwareEngineering

Post #14: Distributed Sagas: Orchestration vs Choreography for Multi-Service Transactions

[Target Chapter: 10 Saga Pattern.html](#)

■ Why 2-Phase Commit (2PC) is an anti-pattern in microservices and how Sagas solve distributed transactions:

In microservices, each service owns its private database. A checkout transaction spans Order Service, Payment Service, and Inventory Service.

■ Why 2PC Fails at Scale:

Two-Phase Commit holds distributed locks across all databases until everyone commits. If 1 service is slow, all database connections stall, causing latency spikes and blocking throughput.

■ The Solution: The Saga Pattern (A sequence of local transactions).

If a step fails, the Saga executes **Compensating Transactions** in reverse order to undo changes!

■ Choreography (Event-Driven):

- Services publish and listen to domain events over Kafka/RabbitMQ.
- Pros: Simple, no central orchestrator.
- Cons: Hard to track workflow state as service count grows (spaghetti dependencies).

■ Orchestration (Command-Driven):

- A central Saga Orchestrator (Temporal, Camunda, AWS Step Functions) sends explicit commands to each service and waits for acknowledgments.
- Pros: Centralized visibility, easy rollback tracking, handles complex timeout retries.

■ Do you use Orchestration or Choreography for distributed checkout flows?

(■ Full live guide in first comment! ■)

#Microservices #SagaPattern #Kafka #DistributedSystems #SoftwareArchitecture #Fintech

Post #15: Transactional Outbox Pattern & Debezium Change Data Capture (CDC)

[Target Chapter: 11 Event-Driven Architecture Pattern.html](#)

■ How to update your SQL database and publish to Kafka without dual-write inconsistency:

The Dual-Write Problem:

```
```python
db.save(order) # Step 1: Succeeds
kafka.publish(order) # Step 2: Fails or network timeout!
```
```

If Step 2 fails, your DB has the order, but downstream services never get notified! If you swap the order, Kafka gets the message but the DB write might fail!

■ The Solution: Transactional Outbox Pattern + Debezium CDC.

1■■ Atomic Local Commit:

In a single ACID database transaction, insert the Order into the `Orders` table AND an event record into an `Outbox` table:

```
```sql
BEGIN;
INSERT INTO orders (id, user_id, amount) VALUES (101, 42, 99.00);
INSERT INTO outbox (event_id, aggregate_type, payload) VALUES (uuid(), 'ORDER_CREATED', json_payload);
```

```
COMMIT;
```
```

2■■ Change Data Capture (Debezium):

Debezium tails the database Transaction Log (PostgreSQL WAL / MySQL binlog) and streams events to Kafka with **At-Least-Once** delivery guarantees and zero application polling overhead!

■ Have you implemented the Outbox pattern with Debezium in production?

(■ Full live guide in first comment! ■)

#Kafka #EventDriven #PostgreSQL #Debezium #Microservices #SystemDesign

■■ Grokking Low Level Design (LLD) with JavaScript 2026

Post #16: SOLID Principles in Modern JavaScript & TypeScript

[Target Chapter: 01 OOP Fundamentals & SOLID Principles in JavaScript.html](#)

■ Mastering the 5 SOLID Principles in Modern JavaScript / TypeScript with real code examples:

Writing clean, maintainable, and extensible code is what separates Junior developers from Senior/Staff engineers.

Here is the quick breakdown of SOLID:

1■■ Single Responsibility (SRP): A class/module should have only ONE reason to change (e.g. separate `InvoiceCalculator` from `InvoicePDFRenderer`).

2■■ Open/Closed (OCP): Open for extension, closed for modification (use Strategy Pattern to add new discount rules without editing core pricing logic).

3■■ Liskov Substitution (LSP): Subclasses must be substitutable for their base types without breaking invariants.

4■■ Interface Segregation (ISP): Clients should not depend on interfaces they do not use (split giant interfaces into focused role contracts).

5■■ Dependency Inversion (DIP): High-level modules should depend on abstractions (interfaces), not concrete implementations (inject DB repositories via constructors).

(Full JavaScript/TypeScript code examples for all 5 principles in the guide!)

■ Which SOLID principle do you see violated most often in code reviews?

(■ Full live guide in first comment! ■)

#JavaScript #TypeScript #SOLID #CleanCode #OOP #WebDevelopment #SoftwareEngineering

Post #17: Designing an In-Memory Multi-Tier Rate Limiter in Node.js

[Target Chapter: 06 Medium LLD & System Architecture Problems.html](#)

■■ Designing a high-performance Token Bucket & Sliding Window Log Rate Limiter in Node.js:

How do you protect your Node.js APIs from brute-force attacks and noisy neighbors without killing throughput?

■ Two Core Algorithms Compared:

1■■ Token Bucket Algorithm:

- Tokens added at fixed refill rate (e.g., 10 tokens/sec) up to capacity \$B\$.
- Each request consumes 1 token.
- Strengths: Memory-efficient (\$O(1)\$ space), allows bursts up to bucket capacity.

2■■ Sliding Window Log Algorithm:

- Tracks exact timestamps of requests in a Redis Sorted Set (`ZSET`).
- Drops entries older than \$(now - windowSize)\$ and counts remaining elements with `ZCARD`.
- Strengths: 100% boundary accuracy, zero burst leakage at boundary edges.
- Tradeoff: Higher memory footprint (\$O(N)\$ elements per user).

(Complete executable Node.js class implementation with Lua atomic scripts in the guide!)

■ Do you run rate limiters at the API Gateway level (Envoy/Kong) or as Node.js middleware?

(■ Full live guide in first comment! ■)

#Nodejs #JavaScript #RateLimiter #Security #Backend #SystemDesign